GEOG0111 – Scientific Computing

PART B – Earth Observation Project

Submission basics:

- Submit Jupyter Notebook (as .pdf) via Moodle.
- You have to run the notebook as whole at once, so the the execution counts should start at number 1 with first code cell and continue incrementaly by 1 throughout the following code cells.
- Do **not** include your name in the notebook; use only your **student number** if needed. When you will use API and logins please made those anonymous as well. For example, for GEE name the project using your student number.

---

1. Overview

In this coursework you will design and implement a self-contained data analysis project using **Python for Earth Observation (EO)** or related environmental data.

You will:

- choose a simple EO problem or question (e.g. vegetation index over an area, urban vs rural temperature difference, simple change detection between two dates, etc.),
- obtain suitable data (raster, vector or tabular),
- process and analyse the data using Python, and
- interpret and communicate your results clearly.

The project must include:

- EO data,
- data with some temporal resolution,
- series of functions

The aim is not to build a complex research experiment, but to demonstrate that you can:

- structure a small project,
- write and run reproducible code, and

- explain what you did and what it means.

---

2. Assessment Criteria

Your work will be marked using **four categories**. Each category contributes by various **%** of the coursework mark.

2.1 Introduction (**15% of the mark**)

We look for:

- A clear and focused question or aim.
- Short background: Why is this interesting / relevant (scientifically or practically)?
- A description of the data you use (source, type, basic characteristics).
- A brief plan of the analysis steps you will perform.

2.2 Data processing (**60% of the mark**)

We look for:

- Clear, well-structured code cells.
- Correct loading of data and basic checks (dimensions, CRS, basic statistics, plots).
- Appropriate processing steps (e.g. subsetting, masking, computing indices, aggregating).
- Use of functions to avoid copy-paste and to show structure.
- Reasonable code style: comments, informative variable names, avoiding unnecessary complexity.

2.3 Interpretation of results (**20% of the mark**)

We look for:

- Clear figures / tables and short captions or explanations.
- Text describing what the results show in relation to your question.
- Awareness of limitations or uncertainty (e.g. data resolution, temporal coverage, simple method).
- Logical connection between the analysis steps and the conclusions you draw.

2.4 Conclusion (**5% of the mark**)

We look for:

- A short summary of what you did and what you found.
- A direct answer to your original question or aim.
- 1–3 ideas for improvement or future extension (e.g. more data, different methods).
- Reflection on what you learned (technical or conceptual).

3. Introduction (**15% of the mark**)

Use this section to explain **what you are doing and why**.

Project title

- Give your project a short, descriptive title (e.g. *"NDVI analysis for Regent's Park"*).

Question or aim

- What is the main question you want to answer, or the main pattern you want to show?
- Keep it focused and realistic for a coursework project.

Background (short)

- Explain why this question is interesting.
- You can mention environmental / scientific relevance in simple terms.

Data description

- What data are you using? Examples:
  - Satellite imagery (e.g. Sentinel-2, Landsat, MODIS).
  - Derived products (e.g. NDVI data, land cover maps).
  - Other environmental or climate data.
- For each dataset, state:
  - **Source** (website, API, etc.).
  - **Type** (raster, vector, tabular) and basic characteristics (resolution, time period).

Plan

- In bullet points, outline the main processing steps you will perform.

(Write your introduction below this line. You can use additional markdown cells if needed.)

4. Data processing (**60% of the mark**)

This section contains the technical work of your project. Your goal is to write clear, well-structured, and reproducible code that another person could understand and reuse.

Your data processing should be organised into multiple cells, each with a clear purpose. Avoid placing everything in a single long cell.

You should normally include:

- code to **load** your data (from file, url or API),
- checks and **basic exploration** (print shapes, simple plots, basic statistics) - basically explore and show the user of the notebook what the data are,
- your **processing steps** (e.g. filtering, masking, computing indices, aggregating over time or space),
- generates results for interpretation,
- save all important files and results in usable formats

Aim for code that another student could read and follow. Add short comments where it helps understanding.

To support good scientific computing practice, you must based your code on functions and define your own functions with appropriate input variables. When functions are used they have to be used in the following workflows.

This means:

- Do not write long sequences of processing directly in the notebook.
- Do break them into functions, for example:
    - load_data(path)
        - compute_index(band_a, band_b)
        - clip_to_aoi(array, aoi)
        - aggregate_monthly(df)
        - and similar.
- Every function must:
    - have clear input arguments (not rely on notebook variables),
        - return a value (not print everything),

    - include a short docstring describing what it does.

Such approach makes your code:

- reusable outside the notebook,
- easier to test,
- easier to understand,
- cleaner and more professional.

> Note: It is fine to keep the analysis simple as long as it is correct and clearly explained.

Visualisation of processed data

Create one or more figures that show the key results of your processing.

Examples:

- A map of an index (e.g. NDVI) for your area of interest.
- A time series plot.
- A comparison between two dates or two locations.

Add a brief description above or below each figure explaining **what it shows**.

---

5. Interpretation of results (**20% of the mark**)

In this section, you move from **"what the code did"** to **"what the results mean"**.

You should:

- Refer to the key figures or tables from the data processing section.
- Describe the main patterns you see.
- Relate these patterns to your original question or aim.
- Mention any limitations or uncertainties (e.g. coarse resolution, clouds, short time period, simple method).

---

6. Conclusion (**5% of the mark**)

Provide a short conclusion that answers these questions:

1. **What did you do?**
   – One or two sentences summarising the data and methods.
2. **What did you find out?**
   – One or two sentences giving the main result(s) in relation to your question or aim.
3. **What are the limitations?**
   – One or two sentences on the main limitations of your analysis.
4. **What could be done next?**
   – 1–3 ideas for how you would extend or improve the work with more time or data.

Try to keep this section **short (around 150–250 words)**, but precise.

---

7. Notes on academic integrity and AI tools

You are allowed to use AI tools (such as ChatGPT, Gemini, GitHub Copilot, etc.) in this coursework as coding assistants. However, here are rules to ensure the work reflects your understanding.

What you may use AI to:

- help write or debug code,
- generate bare code without comments,
- get ideas, explore concepts, or ask for clarification,
- look up syntax or examples,
- help structure a function or workflow.

What AI must not generate:

- comments in your code,
- explanations of your results,

- your introduction, interpretation, or conclusion text,
- any text that replaces your own reasoning.

How to prompt AI for code

If you use AI to produce code, you must include a line in your prompt like:

“Write only Python code with no comments”

This ensures that:

- all comments in the notebook are written by you,
- and that comments reflect your understanding, not AI output.

Your responsibility

All comments, explanations, and written analysis in this notebook must be:

- in your own words,
- based on your own understanding,
- clearly linked to the code and results you produced.

You will be assessed on your explanation and project itself and not solely on how advanced the code is.

AI usage declaration

At the end of the notebook, include a short note describing any AI support you used. For example:

“I used ChatGPT to help create an outline for a function and to debug an error. I wrote all comments and explanations myself.”

This transparency ensures fairness and academic integrity.